Track 1: Merchant Growth AI

Build the AI business partner for every Paytm merchant.

Build scalable AI-led solutions that help Paytm merchants grow their business, manage operations, and serve customers better. Think beyond payments, identify opportunities where AI can become a trusted business copilot for millions of merchants.

CLAUDE: I did one search to see what Paytm has already shipped, because that decides what counts as “generic” for this judging panel.

Assumptions (you didn’t give these, so correct me if they’re wrong): a team of 3–4 students, comfortable with Python, React or Flutter and LLM/vision APIs, with roughly 24–36 hours. I treat the demo as running on synthetic merchant data, since you won’t get real Paytm data.

STEP 1: DEEP ANALYSIS

The problem in simple words
Paytm already helps merchants take payments. Now it wants an AI that helps them run and grow the shop: make more sales, avoid mistakes, get money on time, and stay compliant. It has to work for millions of merchants, most of whom are small, busy, not tech-savvy and often not fluent in English.

Who suffers, and the real root cause

- The sufferer is the 1–5 person kirana, tea stall, salon, pharmacy or repair shop. The owner is also the cashier, buyer, accountant and marketer, and has no time to read dashboards.
- Root cause: these merchants make decisions on instinct, because the information they need is scattered. Some is on paper (khata, supplier bills), some is in their head (who owes them, who is a regular), and some sits with outside parties (weather, festivals, suppliers). Paytm sees only the payment slice of the business.

The hidden need most teams will miss

1. Merchants don’t need insights, they need actions taken for them. “Sales dropped 12%” is useless to someone with 40 customers waiting.
2. The copilot is blind to cash, credit and costs. Paytm sees UPI and cards but not udhaar, cash sales or purchase bills. Without those, any “AI advice” is shallow.

3. Trust and relationships matter more than efficiency. A merchant won't chase a neighbour for ₹800, won't ask a regular to come back, and won't question a supplier. An AI that does the socially awkward part is valuable.
4. Paytm's unfair advantage is that it sits on both sides of the transaction, and across millions of merchants. Solutions that use this are hard to copy. Solutions that only use one merchant's data are not.

What judges will look for

- *Scalable:* Does it work for the 1st merchant and the 10 millionth? Is the cost per merchant sane?
- *Vernacular and voice-first:* Hindi and regional languages, low literacy, and a phone or Soundbox as the interface.
- *Beyond payments:* Credit, inventory, compliance, customers, suppliers.
- *Moat:* Uses Paytm's data, devices, lending arm or two-sided network.
- *Measurable outcome:* ₹ recovered, % revenue lifted, hours saved.
- *Trust and safety:* Explainability, merchant consent, AI errors that don't cost money.
- *Monetization:* Paytm has already launched an AI Soundbox with a fair-use token subscription (an AI token subscription model aimed at retail chains, cafés, and local stores), so judges are already thinking about how AI pays for itself. hdfcsky

Important competitive fact: Paytm's AI Soundbox already has an assistant that talks to merchants in 11 languages and answers questions about payment history and business performance. So "voice assistant that answers questions about your sales" is not novel to Paytm's own judges. Avoid it. fonearena

---

STEP 2: SIMPLE EXAMPLE

Sunita runs a grocery shop in Lucknow. She has ₹46,000 in udhaar written in a notebook across 38 families. She's embarrassed to ask, and some of them are her best customers. Meanwhile she can't afford to restock for Diwali, and she doesn't know that 6 of her regular customers haven't come in for a month.

She doesn't lack payments. She lacks money she's already earned, and the nerve to ask for it. A dashboard won't fix that. A copilot that quietly does the awkward work will.

STEP 3: KILL THE GENERIC

Most teams will submit these five. Don't.

1. **Multilingual business chatbot ("Ask your sales")**
   Paytm's AI Soundbox already does this. You'd be demoing something the judges own.
2. **Sales analytics dashboard with AI summaries**
   Merchants don't read dashboards, and the output is passive information with no action.
3. **Inventory prediction app**
   It needs stock data that merchants don't have, so on synthetic data it looks fake. Every team shows the same line chart.
4. **AI marketing generator (posters, WhatsApp texts, offers)**
   It has no closed loop, so you can't prove it made a rupee. It's also a commodity feature of every LLM.
5. **Merchant loan recommender / credit score**
   It's the most obvious "Paytm" idea, so a quarter of the teams will pitch it with the same XGBoost on fake data.

They fail because they're passive (they inform instead of act), single-source (they use only transaction data), and demo as the same screen.

STEP 4: UNIQUE WINNING SOLUTIONS

1. Udhaar Autopilot (the AI that collects what you're owed, politely)

1. **Core idea:** The merchant photographs the handwritten khata page. AI digitizes it, works out who is likely to pay and how to ask, and sends personalized nudges with a UPI link. When money arrives, the Soundbox announces it.
2. **Different because:** It tackles the social awkwardness of credit, not a data problem. Its input is paper (a data source others will ignore) and its output is a ₹ number.
3. **How it works:**
   1. Merchant snaps a khata page in Hindi or any regional script. A vision LLM extracts name, amount, date and item.
   2. It builds a ledger and a per-customer profile (ageing, amount, past payment behaviour if a UPI history is available).
   3. A recovery-priority model scores customers (amount × likelihood × relationship value).
   4. An LLM drafts a tone-calibrated message: gentle for a loyal regular, firmer for a 90-day-old balance. Output can be a Hindi voice note in the merchant's name, or a WhatsApp or SMS with a UPI deep link.
   5. Payment lands, the ledger updates automatically, and the Soundbox says "Sharma ji ne ₹1,200 chuka diye."

    6. The merchant approves with one tap or “haan” by voice. Nothing is sent without approval.
4. **Tech stack:** React or Flutter PWA, FastAPI, vision LLM for handwriting, a small LLM for tone and language, gTTS or Bhashini for voice, Razorpay or UPI-intent mock for the payment link, SQLite.
5. **Realistic demo:** Real photo of a handwritten khata (have your own team write one) → ledger on screen → ranked list → generated Hindi voice note → “payment” via mocked UPI link → ledger update plus a Soundbox-style announcement. All doable in 24–30 hours.
6. **Impact (illustrative, state it as assumptions):** If a typical small shop carries about ₹40,000 in outstanding credit and the system speeds recovery by 15%, that’s about ₹6,000 of cash freed per merchant. Across 1 million merchants that’s about ₹600 crore of working capital unlocked, with no loan needed.
7. **Why judges select it:** It’s emotionally instantly understood, it has a visible money outcome, the demo ends with a payment landing, and it opens a path to Paytm’s lending and Soundbox business.
8. **Biggest risk:** Handwriting OCR errors and the optics of “AI debt collection.” Handle it by showing a confirm-before-save screen for extracted entries, keeping every nudge merchant-approved, and building the tone engine around preserving the relationship, not pressure.

2. Regulars Radar (win back the customers who quietly stopped coming)

1. **Core idea:** Detect a merchant’s repeat customers and spot who has slipped away. Then automatically send them a small, personalized comeback offer through the Paytm app.
2. **Different because:** It uses Paytm’s two-sided network. Only Paytm knows both the shop and the shopper, so a competitor or any hackathon team with just a merchant dataset can’t copy it.
3. **How it works:**
    1. From payment history, cluster repeat visitors per shop and learn each one’s rhythm (e.g., every 9 days).
    2. Flag anyone more than 2× overdue as lapsing.
    3. An LLM generates an offer sized to the shop’s margin (e.g., “free chai with your next order”).
    4. Opt-in push goes to the customer in the Paytm app. The merchant sees a one-line summary.
    5. Redemption is tracked, and the Soundbox says “Sunita ji wapas aayi.”
4. **Tech stack:** Python (pandas, scikit-learn or lifetimes library for churn), FastAPI, React for merchant and customer phone mock, an LLM for offer copy.

5. **Realistic demo:** Synthetic transaction dataset of 5,000 customers → lapse detection → two phone mockups side by side (merchant and customer) → redemption closes the loop.
6. **Impact (illustrative):** If repeat customers drive about 60% of revenue and you win back just 5% of lapsed regulars, that's roughly a 2–4% revenue lift per merchant with near-zero effort.
7. **Why judges select it:** It's a clear, scalable growth feature, and it shows the Paytm moat.
8. **Biggest risk:** Privacy ("you're targeting customers with their payment data"). Handle it with opt-in "follow this shop" consent, no customer data exposed to the merchant, and a tight privacy slide in the pitch.

3. Twin Shop Playbook (learn from the best shop that looks just like yours)

1. **Core idea:** Find the top-performing merchants that most resemble you (category, area type, size). Then tell you the one or two behaviours that explain the gap.
2. **Different because:** It's cross-merchant intelligence, not single-merchant analytics. It converts Paytm's scale into a personal playbook ("shops like yours earn 18% more because they open at 6 am for the milk and tea crowd").
3. **How it works:**
    1. Embed each merchant by category, location tier, ticket-size distribution and hourly pattern.
    2. For a target merchant, retrieve lookalikes and split them into top and bottom quartile.
    3. Diff the behaviours: opening hours, card acceptance, average ticket, product mix, peak coverage.
    4. An LLM explains the top 1–2 differences in plain Hindi with an estimated ₹ upside.
    5. The merchant gets one actionable "try this this week."
4. **Tech stack:** Python, sentence embeddings or simple feature vectors, k-NN, an LLM for the narrative, Streamlit or React.
5. **Realistic demo:** A synthetic dataset with planted real patterns (e.g., stalls that stay open later perform better) → pick a merchant → show lookalikes → one recommendation with the gap quantified.
6. **Impact (illustrative):** If 20% of merchants act on a recommendation and see a 5% revenue lift, that's meaningful at scale. Treat as directional, not guaranteed.
7. **Why judges select it:** It shows a deep grasp of the "scale as moat" idea, and judges instantly understand "shops like yours do this."
8. **Biggest risk:** Correlation presented as causation. Handle it by labelling suggestions "worth testing," showing the lookalike sample size, and tracking before and after.

4. Cash Crunch Radar (a forecast that warns you 10 days before you run out of money)

1. **Core idea:** Predict when a merchant's cash will run short (rent, festival stock, supplier dues) and offer a small loan sized to that gap, repaid as a small slice of daily settlements.
2. **Different because:** Everyone pitches "credit score." This sells the timing and the shape: a warning plus a right-sized offer, not a generic loan.
3. **How it works:**
    1. Forecast daily inflows from settlement history, seasonality and festival calendar.
    2. Subtract known outflows (rent, EMI, recurring supplier payments).
    3. Detect a projected shortfall window.
    4. Generate a loan quote sized to the gap, repaying a fixed % of daily collections.
    5. Notify by voice: "Agle hafte ₹18,000 ki kami aa sakti hai."
4. **Tech stack:** Python, Prophet or a simple gradient-boosted forecaster, FastAPI, React, an LLM for the explanation.
5. **Realistic demo:** Synthetic 12-month history → forecast chart with a shaded shortfall → loan quote card.
6. **Impact (illustrative):** Avoids missed supplier payments and emergency borrowing from informal lenders, which is often at far higher rates.
7. **Why judges select it:** It maps directly to Paytm's lending business, with clean monetization.
8. **Biggest risk:** Forecast accuracy and responsible lending. Handle it with prediction ranges (not a single number), a conservative loan cap, and an explicit "why am I seeing this" screen.

5. Sarkari Chitthi Decoder (photograph any government notice, get a plan)

1. **Core idea:** The merchant photographs a GST notice, municipal letter, FSSAI renewal or electricity bill. AI explains it in plain words, gives the deadline, drafts the reply and offers one-tap payment where relevant.
2. **Different because:** It tackles fear and compliance, a domain almost no team will touch. It is "beyond payments" in the most literal sense.
3. **How it works:**
    1. Merchant uploads a photo of the notice.
    2. Vision LLM extracts issuer, type, amount, deadline and demanded action.
    3. A grounded explanation in the merchant's language ("This is a late-filing penalty of ₹X; reply by 12 Nov").
    4. Draft reply or checklist of documents.
    5. If a payment is due, one-tap Paytm payment.
4. **Tech stack:** Vision LLM with a structured output schema, a small knowledge base of common notice types (RAG), React, FastAPI.

5. **Realistic demo:** Three sample notices (GST, municipal, FSSAI) → plain-language card with deadline and action → drafted reply → payment screen.
6. **Impact (illustrative):** Avoids penalties and missed deadlines; the benefit is trust more than direct ₹.
7. **Why judges select it:** It's memorable, emotionally resonant, and shows range beyond payments.
8. **Biggest risk:** Wrong legal interpretation. Handle it with a mandatory "verify with your CA" step, retrieval-grounded answers only, and a limited notice scope.

6. Dukaan Mausam (tomorrow's shop forecast from rain, festivals and local events)

1. **Core idea:** Each morning the merchant gets a 20-second voice brief: "Kal baarish hai, chai aur pakode zyada rakho. Mohalla mein shaadi hai, cold drinks stock karo."
2. **Different because:** It uses external signals others ignore (weather, festival calendar, local events) combined with the merchant's own history.
3. **How it works:**
    1. Pull weather forecast, festival calendar and local event data.
    2. Learn each shop category's sensitivity (via history on synthetic data).
    3. Generate a ranked "prepare for tomorrow" list.
    4. Deliver as a short voice note.
4. **Tech stack:** Open-Meteo API, Python, an LLM, gTTS, a simple React app.
5. **Realistic demo:** Pick a shop category and city, show tomorrow's brief with reasoning and voice playback.
6. **Impact (illustrative):** Reduces stockouts and waste; modest but easy to explain.
7. **Why judges select it:** It's easy to demo and pleasant to watch.
8. **Biggest risk:** Looks like a lightweight weather app. Handle it by quantifying the effect from your own history (e.g., "rainy days raised tea sales 31%") and tying it to a concrete action.

7. Mandi Pool (neighbouring shops buy together to cut wholesale costs)

1. **Core idea:** AI groups nearby merchants' restocking needs into one bulk order and negotiates better supplier prices.
2. **Different because:** It flips the problem from "find more customers" to "improve margin on the buy side," and changes who the merchant is talking to (suppliers).
3. **How it works:**
    1. Estimate each nearby shop's weekly needs.
    2. Cluster by locality and category.

    3. Build a consolidated purchase order.
    4. Get supplier quotes, compare, and share cost splits.
    5. Merchants confirm by voice; payment goes through Paytm.
4. **Tech stack:** Python, clustering, LLM for supplier-quote parsing, React, mock supplier API.
5. **Realistic demo:** 10 nearby shops → pooled PO → price saving shown versus buying alone.
6. **Impact (illustrative):** If pooling saves 3–5% on purchases, that's a large share of a thin-margin shop's profit.
7. **Why judges select it:** It's a smart marketplace-like angle with network effects.
8. **Biggest risk:** Cold-start problem on the supply side and hard to demo honestly. Handle it by scoping to one category and clearly labelling supplier data as simulated.

STEP 5: RANKING AND FINAL PICK

Rank	Solution	Why it lands here
1	**Udhaar Autopilot**	Highest emotional clarity, a visible ₹ outcome, a demo that ends in a payment, feasible in your time, and unexpected input (paper khata).
2	Regulars Radar	Strongest Paytm-moat story and a clean growth loop, but privacy questions and heavier on synthetic data.
3	Twin Shop Playbook	Impressive "scale as moat" story, but the demo is less dramatic and causal claims are risky.
4	Cash Crunch Radar	Great monetization link, but closest to the "credit" idea other teams will pitch.
5	Sarkari Chitthi Decoder	Memorable and different, but legal-accuracy risk and weaker link to growth.
6	Dukaan Mausam	Easy and pleasant, but may read as a weather add-on.
7	Mandi Pool	Smart idea, hardest to demo honestly.

Build this: **Udhaar Autopilot**

Why this one

- **It answers the hidden need.** It does the socially awkward work for the merchant and ends in cash, not insight.

- **It closes a data gap Paytm has.** Paytm sees UPI but not credit sales. A photo of the khata page brings the missing piece into the system.
- **Its demo is a story.** Handwritten page → ledger → ranked list → Hindi voice note → payment lands → Soundbox announces. The judges watch money arrive.
- **It's differentiated from the other 80%.** Others will demo chatbots and dashboards; you'll demo a handwritten notebook turning into recovered cash.
- **It opens a business path.** Freed working capital reduces borrowing need, and recovery data becomes a credit signal for Paytm's lending.

How to execute it so it wins

1. Build the end-to-end loop before polishing anything. A rough version of every step beats a perfect OCR screen.
2. Show the confirm-before-save step. It signals responsible AI.
3. Keep every outbound message merchant-approved, and say so on a slide.
4. Open the pitch with Sunita and the ₹46,000, then switch to the live demo within 30 seconds.
5. Close with one number: ₹ of cash unlocked per 1 million merchants, with assumptions stated openly.

```
**GPT**:
```

1. Problem Analysis

Goal: Build an AI business partner that helps Paytm merchants increase profits, manage operations, and serve customers better.

The real problem

Small merchants often struggle to predict demand, manage inventory, retain customers, and make profitable decisions.

Hidden need: Merchants don't need another dashboard. They need AI that identifies problems and recommends specific actions to improve their business.

What judges may look for

- A genuine merchant problem.
- Originality beyond basic chatbots and dashboards.

- Meaningful use of AI.
- A working prototype.
- Measurable business impact.
- Scalability across merchant categories.

2. Simple Example

A grocery shop has 30 packets of snacks that aren't selling, while another popular product frequently runs out.

A basic app shows sales figures. An AI business partner identifies the slow-moving stock, recommends a suitable promotion, and suggests when to replenish the popular product.

3. Five Generic Ideas to Avoid

1. Basic AI chatbot.
2. Sales analytics dashboard.
3. Generic product recommendation system.
4. AI social media content generator.
5. Basic inventory management application.

These ideas can be useful, but they need a distinctive feature to stand out.

4. Seven Unconventional Solutions

1. StockSaviour AI — Inventory Recovery Engine

Identifies slow-moving products and recommends ways to recover their value through bundles, discounts, or adjusted purchase orders.

Differentiator: Focuses on recovering money tied up in inventory rather than simply tracking stock.

Demo: Mobile application showing inventory risks, recovery recommendations, estimated margins, and simulated outcomes.

Risk: Could look like ordinary inventory software unless the recommendation engine is the central feature.

2. MarginGuard — Profit Protection AI

Analyzes product costs, selling prices, and discounts to identify unprofitable decisions.

Differentiator: Optimizes profitability rather than just revenue.

Demo: Compare gross margins before and after a proposed discount.

Risk: Could look like a calculator without automated insights.

3. BazaarBridge — Merchant-to-Merchant Stock Matching

Connects merchants with excess inventory to other merchants who need those products.

Differentiator: Creates a potential local merchant network instead of solving inventory problems one shop at a time.

Demo: Simulate five shops and demonstrate profitable inventory matches.

Risk: Real-world success depends on merchant participation and product compatibility.

4. DemandPulse — Context-Aware Demand Prediction

Combines sales history with relevant external signals, such as weather or holidays, to estimate upcoming demand.

Differentiator: Uses context to improve purchasing and preparation decisions.

Demo: Compare demand scenarios for normal days, rainy days, and festivals.

Risk: External signals must meaningfully improve the forecast.

5. ShockShield — Business Disruption Predictor

Detects unusual sales patterns, supplier delays, and potential stockouts, then recommends corrective actions.

Differentiator: Focuses on preventing operational problems before they become expensive.

Demo: Simulate a supplier delay and identify products likely to run out.

Risk: Could become a basic alerting dashboard without actionable recommendations.

6. VoiceLedger — Voice-Powered Business Assistant

Allows merchants to record sales, update stock, and request reorder recommendations through natural-language instructions.

Differentiator: Converts voice commands into validated business operations.

Demo: Speak an instruction, confirm the proposed changes, and update the database.

Risk: Voice interaction alone isn't sufficiently distinctive.

7. SilentDrop — Customer Retention AI

Identifies customers whose purchasing frequency has declined and suggests targeted retention strategies.

Differentiator: Detects potentially lost repeat business before merchants notice it.

Demo: Use simulated customer purchase histories to identify declining engagement.

Risk: Requires suitable customer data and appropriate privacy safeguards.

5. Final Ranking

Rank	Solution	Main advantage
1	**StockSaviour AI**	Strong balance of originality, feasibility, and measurable impact
2	MarginGuard	Easy to build and financially convincing
3	BazaarBridge	Distinctive network-based approach
4	DemandPulse	Strong predictive AI use case
5	ShockShield	Proactive operational problem-solving
6	VoiceLedger	Practical, interactive demonstration
7	SilentDrop	Useful retention insights but data-dependent

This ranking reflects strategic judgment, not measured probabilities of winning.

6. Final Recommendation: StockSaviour AI

The idea

Build an **AI Inventory Recovery Engine** that identifies slow-moving inventory and recommends the most economically sensible action to recover its value.

How it works

1. Import sales and inventory data.
2. Identify products at risk of remaining unsold.
3. Calculate inventory cost and gross margins.
4. Compare discounts, bundles, and adjusted purchasing decisions.
5. Recommend an action with transparent financial estimates.
6. Track the outcome using subsequent sales data.

Suggested tech stack

- **Frontend:** Flutter
- **Backend:** Python and FastAPI
- **Database:** Firebase or SQLite
- **Data analysis:** pandas
- **AI:** LLM API for explanations
- **Decision engine:** Python business logic

Why choose it?

- Solves a specific merchant problem.
- Can be demonstrated without proprietary Paytm data.
- Combines AI with verifiable financial calculations.
- Offers measurable outcomes, such as inventory cost recovered and gross profit.
- Can expand to grocery stores, clothing shops, and other retailers.

Final strategy

Don't build another inventory dashboard. Build a system that compares possible recovery actions and explains which one makes the most financial sense.

That is the feature most likely to make the concept stand out.